Ejercicios APE 5 – Manejo de ficheros en Java

Estudiante: Antony Sebastian Casillas Ochoa

Universidad Técnica de Ambato – Carrera de Software

Ejercicio 1. Crear un fichero en Java

Enunciado:

Desarrollar un programa en Java que permita crear un archivo de texto llamado curso.txt.

Explicación de la solución:

Se declara una constante con el nombre curso.txt y se crea un objeto File con esa ruta. El método createNewFile() devuelve true cuando el archivo es nuevo y false si ya existía, y según ese valor se imprime un mensaje distinto. También se muestra la ubicación completa con getAbsolutePath(). La IOException que puede lanzar createNewFile() se captura con try-catch.

Código fuente (Ejercicio1CrearArchivo.java):

```
import java.io.File;
import java.io.IOException;

public class Ejercicio1CrearArchivo {

    private static final String RUTA = "curso.txt";

    public static void main(String[] args) {
        File documento = new File(RUTA);

        try {
            boolean fueCreado = documento.createNewFile();

            if (fueCreado) {
                System.out.println("[OK] Se creó el archivo: " + documento.getName());
            } else {
                System.out.println("[AVISO] El archivo " + documento.getName() + " ya
estaba creado.");
            }
            System.out.println("Ubicación: " + documento.getAbsolutePath());

        } catch (IOException ex) {
            System.out.println("[ERROR] No fue posible crear el archivo: " +
ex.getMessage());
        }
    }
}
```

Resultado de la ejecución:

```
--- Primera ejecución ---
[OK] Se creó el archivo: curso.txt
Ubicación: C:\Users\Estudiante\APE5\curso.txt

--- Segunda ejecución ---
[AVISO] El archivo curso.txt ya estaba creado.
Ubicación: C:\Users\Estudiante\APE5\curso.txt
```

Ejercicio 2. Escribir información en un fichero

Enunciado:

Desarrollar un programa en Java que permita ingresar un mensaje por teclado y almacenarlo en un archivo de texto existente llamado estudiantes.txt.

Explicación de la solución:

El texto se lee con Scanner dentro de un try-with-resources, que cierra el recurso automáticamente. El guardado se separa en el método guardarTexto(), que abre estudiantes.txt con un BufferedWriter sobre un FileWriter en modo append (parámetro true), de modo que lo nuevo se añade al final sin borrar lo anterior. Se escribe el texto con write() y se agrega el salto de línea con newLine().

Código fuente (Ejercicio2EscribirArchivo.java):

```
import java.io.BufferedWriter;
import java.io.FileWriter;
import java.io.IOException;
import java.util.Scanner;

public class Ejercicio2EscribirArchivo {

    private static final String RUTA = "estudiantes.txt";

    public static void main(String[] args) {
        try (Scanner entrada = new Scanner(System.in)) {
            System.out.print("Ingrese el texto a guardar: ");
            String texto = entrada.nextLine();

            guardarTexto(texto);
            System.out.println("[OK] El texto se añadió al final de " + RUTA);
        } catch (IOException ex) {
            System.out.println("[ERROR] No se pudo escribir en el archivo: " +
ex.getMessage());
        }
    }

    // El segundo parámetro (true) activa el modo "append": conserva el contenido previo
    private static void guardarTexto(String texto) throws IOException {
        try (BufferedWriter salida = new BufferedWriter(new FileWriter(RUTA, true))) {
            salida.write(texto);
            salida.newLine();
        }
    }
}
```

Resultado de la ejecución:

```
Ingrese el texto a guardar: Hola, esta es mi primera línea
[OK] El texto se añadió al final de estudiantes.txt

Ingrese el texto a guardar: Segunda línea de prueba
[OK] El texto se añadió al final de estudiantes.txt

Contenido de estudiantes.txt:
Hola, esta es mi primera línea
Segunda línea de prueba
```

Ejercicio 3. Eliminar un fichero en Java

Enunciado:

Desarrollar un programa en Java que permita eliminar el archivo de texto estudiantes.txt.

Explicación de la solución:

Primero se comprueba con exists() si estudiantes.txt está presente; si no existe, se avisa y el programa termina. Si existe, delete() lo elimina y devuelve true cuando lo logra; en caso contrario se informa que pudo faltar permiso o que otro programa lo tiene abierto.

Código fuente (Ejercicio3BorrarArchivo.java):

```
import java.io.File;

public class Ejercicio3BorrarArchivo {

    private static final String RUTA = "estudiantes.txt";

    public static void main(String[] args) {
        File objetivo = new File(RUTA);

        if (!objetivo.exists()) {
            System.out.println("[AVISO] El archivo " + RUTA + " no existe, no hay nada que
borrar.");
            return;
        }

        if (objetivo.delete()) {
            System.out.println("[OK] Se eliminó el archivo " + RUTA);
        } else {
            System.out.println("[ERROR] No se pudo eliminar " + RUTA
                    + ". Revise los permisos o si está abierto en otro programa.");
        }
    }
}
```

Resultado de la ejecución:

```
--- Primera ejecución ---
[OK] Se eliminó el archivo estudiantes.txt

--- Segunda ejecución ---
[AVISO] El archivo estudiantes.txt no existe, no hay nada que borrar.
```

Conclusiones

1. La clase File representa un archivo en el sistema y ofrece métodos como createNewFile(), exists(), getAbsolutePath() y delete() para crearlo, comprobarlo, ubicarlo y eliminarlo.
2. Combinar FileWriter en modo append con BufferedWriter permite agregar líneas al final de un archivo sin perder su contenido, y el try-with-resources garantiza que se cierre y los datos queden guardados.
3. Capturar la IOException y verificar antes con exists() hace que el programa responda con mensajes claros en lugar de detenerse de forma inesperada.